

Views, Identity, and Safe Change

Expose a stable interface, inspect dependencies, and migrate without guessing.

A view can separate a stable interface from changing storage

Clients need a contract, not permission to depend on every table detail.

NAME

A view gives a query a reusable database object name.

SHAPE

Selected columns and aliases define an interface.

BOUNDARY

Permissions and dependencies can target the view instead of base tables.

Identity changes what the same SQL is allowed to do

Administrative context

- May own objects or hold broad privileges
- Can bypass restrictions that ordinary clients face
- Useful for setup, weak as application-access evidence

Application context

- Uses a specific role or token-derived identity
- Receives only granted operations and policy-visible rows
- Must be tested through its real boundary

Interrogate the execution context before blaming the query

```
SELECT current_database(),  
       current_user,  
       current_schema(),  
       current_setting('search_path');
```

```
SELECT table_schema, table_name  
FROM information_schema.views  
WHERE table_schema = 'metro_support';
```

Context evidence

- Confirm database and identity.
- Record schema resolution rules.
- List the interface objects that actually exist.

A view states its result grain and column contract

```
CREATE VIEW metro_support.active_ticket_summary AS
SELECT t.ticket_id,
       t.subject,
       t.priority,
       u.display_name AS requester_name
FROM metro_support.tickets AS t
JOIN metro_support.users AS u
  ON u.user_id = t.requester_id
WHERE t.status IN ('new', 'open', 'in_progress');
```

Contract

- One row per active ticket
- Stable client-facing column names
- Relationship and status definition remain explicit

Lab 1: Create and verify one stable view

Record execution identity, create a client-facing query interface, and test its result contract.

- Record database, current user, schema, and search path.
- Create one view with an explicit result grain and stable column names.
- Query it under the intended context and verify identifiers, columns, and count.

SUBMIT ONE THING / one view-and-context evidence record

Change compatibility before removing the old path

A migration is a sequence of observable states, not one dramatic command.

Expand, migrate, verify, then contract

Each state should remain understandable and recoverable before the next transition.



A safe change record makes the transition reviewable

```
-- Precondition
SELECT count(*) FROM metro_support.tickets
WHERE category IS NULL;

-- Expand
ALTER TABLE metro_support.tickets
ADD COLUMN category_label text;

-- Migrate
UPDATE metro_support.tickets
SET category_label = initcap(category);

-- Verify
SELECT count(*) FROM metro_support.tickets
WHERE category_label IS NULL;
```

Still missing

- Dependency and client checks
- Rollback or forward-repair decision
- Contract only after old usage stops

Rollback and forward repair solve different situations

Rollback

- Return to a known earlier state
- Best when the transition is still isolated
- May be unsafe after new writes depend on the change

Forward repair

- Correct the current state without reversing history
- Useful after adoption or irreversible transformation
- Requires its own precondition and verification

A five-part change record keeps the claim bounded

- 1 Precondition: the starting schema, data, identity, and dependencies match expectations.

- 2 Change: one controlled transition with its scope and transaction behavior.

- 3 Verification: metadata, data, and representative client behavior are checked.

- 4 Recovery: rollback or forward repair is selected with one remaining risk.

Lab 2: Perform one reversible migration

Inspect the starting state, make one compatible change, verify it, and document recovery.

- Write and run the metadata/data precondition before changing anything.
- Apply one expand-and-migrate sequence in the disposable schema.
- Verify the new state and record rollback or forward repair plus one risk.

SUBMIT ONE THING / one five-part migration record

Safe change preserves a contract while the implementation moves

Know the identity, inspect dependencies, change in stages, and verify through the client boundary.

- What must be true before the change?

- Which old and new paths coexist?

- When is rollback no longer the safest recovery?